

AI-Powered Automated Grading

Tổng quan kỹ thuật & các quyết định thiết kế

Khang Nguyen

Ngày 21 tháng 8 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày các quyết định kỹ thuật đứng sau một hệ thống chấm điểm tự động bài tập lập trình dùng LLM dưới 7 tỷ tham số, xây dựng quanh một yêu cầu cứng, có thể kiểm chứng: *điểm số khớp tuyệt đối (exact-match) giữa các lần chạy lặp lại trên cùng một đầu vào*, kể cả dưới điều kiện serving có batching thật. Đây là bản mở rộng có sơ đồ của docs/TECHNICAL_OVERVIEW.md trong repo dự án, viết cho mục đích portfolio/CV. Phần khung học thuật của cùng dự án (tổng quan tài liệu, mô tả phương pháp chính thức, kết quả kiểu luận văn) nằm riêng ở docs/thesis/.

Mục lục

1 Vấn đề kỹ thuật thực sự	2
2 Vì sao “cứ hỏi LLM chấm điểm” không đáp ứng được yêu cầu này	2
3 Kiến trúc	2
4 Các quyết định kỹ thuật cốt lõi	4
4.1 Correctness không bao giờ chạm vào LLM	4
4.2 Giải mã ràng buộc theo schema, không phải “xin JSON một cách lịch sự”	4
4.3 Một lỗi trong công thức ánh xạ điểm đáng biết	4
4.4 Tính xác định được đo, không giả định	4
4.5 Triển khai xoay quanh giới hạn phần cứng, không bắt chấp nó	5
5 Kết quả định lượng	5
6 Stack công nghệ	6
7 Thực hành kỹ thuật đáng chú ý	6
8 Những gì cố tình nằm ngoài phạm vi (và vì sao đó là một quyết định)	7

Danh sách hình vẽ

1	Mức độ đồng thuận với nhãn tham chiếu tăng đơn điệu theo từng bước phân rã: một lệnh gọi free-form → correctness từ sandbox + một lệnh gọi free-form cho phần còn lại → correctness từ sandbox + một lệnh gọi có cấu trúc cho mỗi tiêu chí rubric. Bảng đầy đủ kèm p-value trong docs/thesis/results.md.	3
2	Toàn bộ pipeline chấm điểm. Correctness chạy trên đường thực thi thật, không có LLM tham gia; mọi tiêu chí rubric còn lại là một lệnh gọi LLM hẹp, ràng buộc theo schema; điểm cuối cùng được tính bằng Python thuần, không bao giờ do mô hình tính.	3

3	Giải mã tự do (trên) để lại một bề mặt mở cho lỗi parse và phương sai điểm số. Giải mã ràng buộc theo văn phạm với một schema nhỏ (dưới) loại bỏ bề mặt đó về mặt cấu trúc — đây là cơ chế đứng sau phương sai bằng 0 trên từng check trong thực nghiệm consistency (Hình 4), không chỉ là ý định thiết kế.	4
4	Phương pháp đo consistency: cùng một mẫu cố định được chấm 100 lần dưới cả serving không batching và có batching. Kết quả trên 4 mẫu dễ và 4 mẫu cố tình khó (có lỗi / trường hợp biên): tỷ lệ exact-match = 1.0 ở mọi điều kiện, phương sai bằng 0 trên mọi breakdown theo từng check. Xem docs/thesis/results.md để biết các lưu ý về phạm vi (cơ chế batching cụ thể của llama.cpp, một mức concurrency).	5
5	Việc chuyển serving engine bị ép buộc bởi phần cứng, nhưng đó cũng chính là điều làm cho giải mã ràng buộc theo schema ở Hình 3 khả thi trên máy này — src/grader/llm_client.py không cần thay đổi gì ngoài base_url vì cả hai engine đều phơi bày API tương thích OpenAI.	6
6	Stack phân lớp, lớp trên sử dụng lớp dưới.	6

1 Vấn đề kỹ thuật thực sự

“Chấm code bằng LLM” tự bản thân nó không phải bài toán kỹ thuật khó — một prompt đơn giản làm được việc đó trong một buổi chiều. Bài toán mà dự án này thực sự giải quyết là:

*Với cùng một đề bài, cùng một rubric, và cùng một bài nộp, hệ thống có thể được thiết kế để luôn trả về **đúng cùng một điểm số ở mọi lần chạy**, kể cả dưới một cấu hình serving có batching như trong sản xuất thật — và tuyên bố đó có thể được chứng minh bằng một phép đo, chứ không phải một cờ cấu hình?*

Đó là bài toán tính xác định (determinism) dưới điều kiện serving thật. Nó cắt ngang qua ba lớp thường không được xem là cùng một bài toán: cấu trúc đầu ra của mô hình, chiến lược giải mã (decoding), và cơ chế batching nội bộ của serving engine.

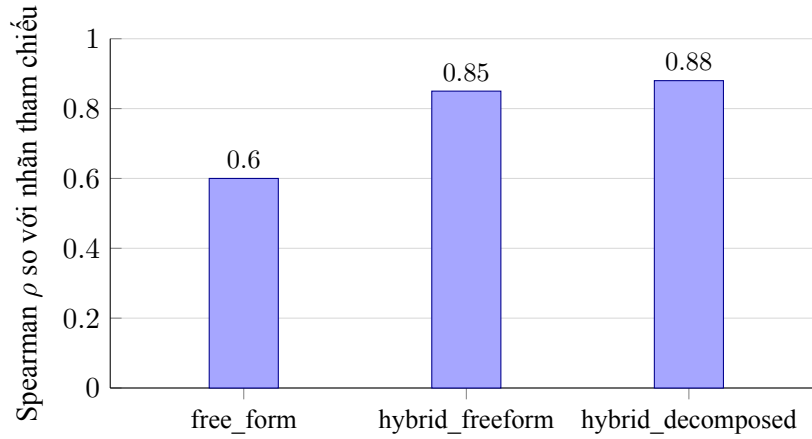
2 Vì sao “cứ hỏi LLM chấm điểm” không đáp ứng được yêu cầu này

Một lệnh gọi tự do (free-form) yêu cầu mô hình trả về một điểm toàn thể 0–100 duy nhất có phương sai giữa các lần chạy rất cao: kể cả ở temperature=0, mô hình vẫn có nhiều đường suy luận văn bản tự do khác nhau nhưng đều hợp lý để đi đến cùng một kết luận, và đường nào được chọn lại nhạy cảm với nhiễu ở tầng serving. Dự án đo trực tiếp khoảng cách này thay vì tin theo cảm tính. Thực nghiệm ablation (experiments/ablation.py) so sánh ba kiến trúc trên cùng bộ dữ liệu 65 mẫu, chấm theo độ tương quan hạng Spearman với nhãn tham chiếu của bộ dữ liệu.

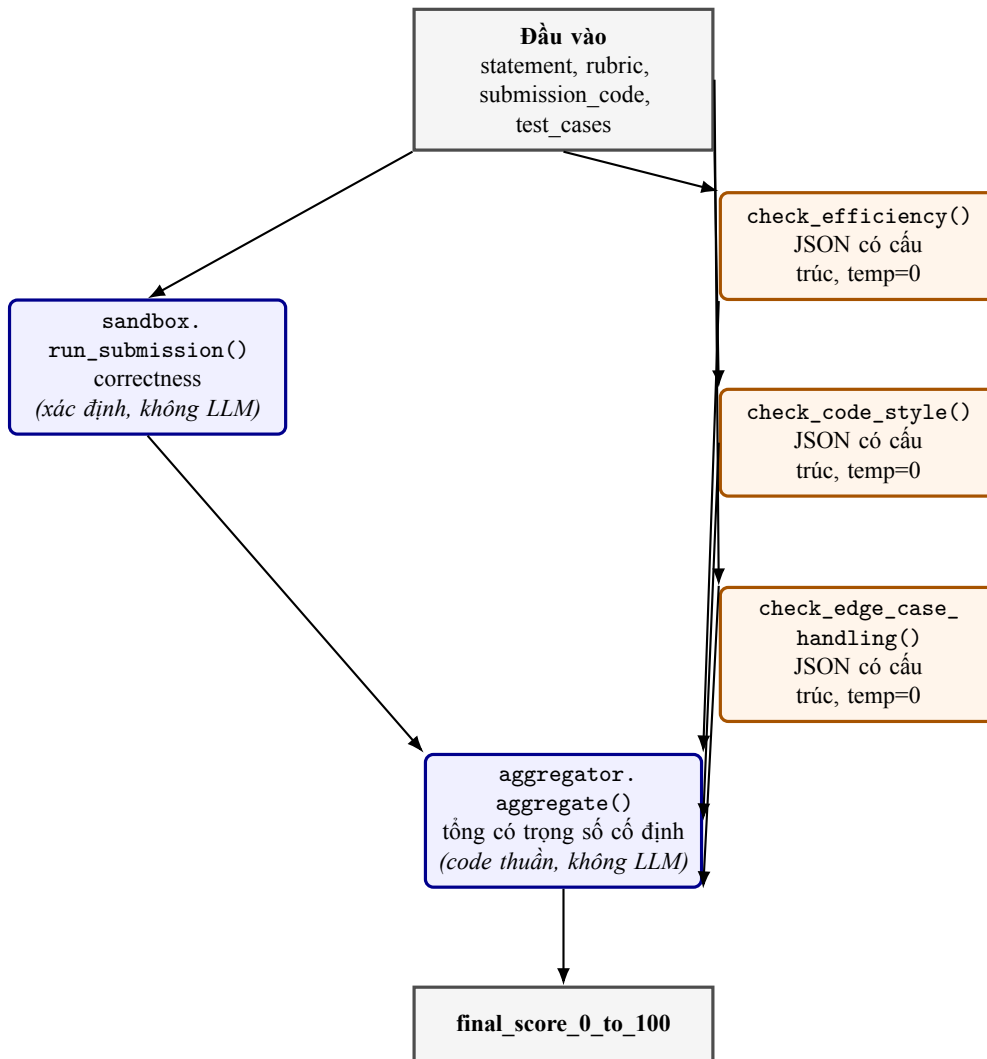
Hình 1 là căn cứ thực nghiệm cho kiến trúc ở mục sau — một kết quả đo được, không phải một sở thích thiết kế.

3 Kiến trúc

src/grader/pipeline.py là một hàm tuần tự duy nhất — cốt lõi **không** xây trên một agent framework (LangChain, CrewAI, v.v.). Không có khám phá nhiều bước hay định tuyến công cụ nào cần điều phối ở đây: chỉ có ba lệnh gọi phân loại độc lập và một phép cộng số học. Mỗi lớp điều phối thêm vào là một nơi có thể làm rõ ràng phi-xác-định hoặc token, mà không mang lại lợi ích gì cho khối lượng công việc này.



Hình 1: Mức độ đồng thuận với nhãn tham chiếu tăng đơn điệu theo từng bước phân rã: một lệnh gọi free-form $\rightarrow$ correctness từ sandbox + một lệnh gọi free-form cho phần còn lại $\rightarrow$ correctness từ sandbox + một lệnh gọi có cấu trúc cho mỗi tiêu chí rubric. Bảng đầy đủ kèm p-value trong docs/thesis/results.md.



Hình 2: Toàn bộ pipeline chấm điểm. Correctness chạy trên đường thực thi thật, không có LLM tham gia; mọi tiêu chí rubric còn lại là một lệnh gọi LLM hẹp, ràng buộc theo schema; điểm cuối cùng được tính bằng Python thuần, không bao giờ do mô hình tính.

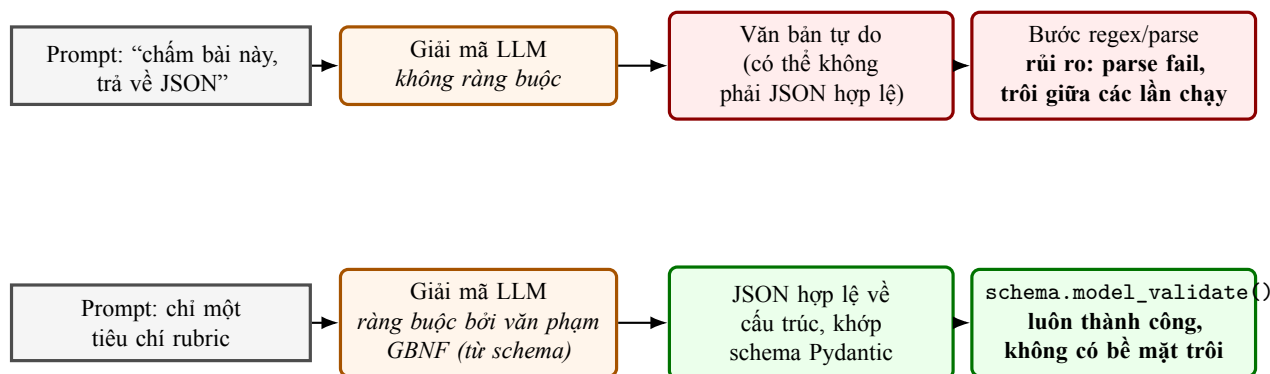
4 Các quyết định kỹ thuật cốt lõi

4.1 Correctness không bao giờ chạm vào LLM

`src/grader/sandbox.py` chạy code nộp bài với các test case thật trong một namespace riêng biệt và tính `pass_rate` trực tiếp — không có suy luận mô hình nào trên con đường quan trọng nhất đối với tính xác định. Điều này cũng làm lộ ra một bài học tinh tế hơn trong quá trình xây bộ dữ liệu (`data/script.md`): hai bài nộp *được thiết kế* để chứa lỗi vẫn đạt 100% test, vì bộ test không tính cờ báo phủ đúng dạng lỗi đó. Correctness từ sandbox chỉ đáng tin cậy bằng bộ test đứng sau nó.

4.2 Giải mã ràng buộc theo schema, không phải “xin JSON một cách lịch sự”

Mỗi tiêu chí rubric ngoài correctness có một prompt hẹp riêng (`prompts/*.txt`) và một schema Pydantic nhỏ riêng (`src/grader/schemas.py`): một `EfficiencyCheck` (boolean + một cụm từ ngắn mô tả mẫu quan sát được), một `StyleCheck` (một enum thứ bậc 3 giá trị), một `EdgeCaseCheck` (boolean + danh sách trường hợp thiếu, có thể rỗng). Các ràng buộc này được áp dụng ngay ở tầng giải mã qua `response_format: json_schema`, được biên dịch thành văn phạm GBNF bởi `llama.cpp` (`src/grader/llm_client.py`) — mô hình *không có khả năng cấu trúc* để sinh ra bất cứ thứ gì ngoài schema, chứ không chỉ đơn thuần được prompt để làm vậy.



Hình 3: Giải mã tự do (trên) để lại một bề mặt mở cho lỗi parse và phương sai điểm số. Giải mã ràng buộc theo văn phạm với một schema nhỏ (dưới) loại bỏ bề mặt đó về mặt cấu trúc — đây là cơ chế đứng sau phương sai bằng 0 trên từng check trong thực nghiệm consistency (Hình 4), không chỉ là ý định thiết kế.

4.3 Một lỗi trong công thức ánh xạ điểm đáng biết

Các điểm thành phần là hàm xác định của đầu ra có cấu trúc, ví dụ:

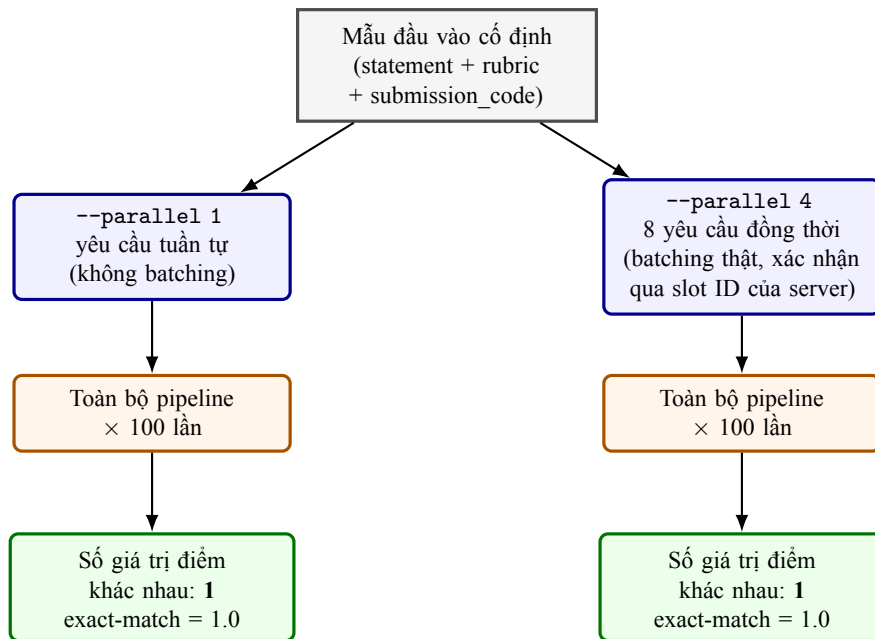
```
# edge_case_handling: handled=False must never silently score 1.0
score = max(0.25, 1.0 - 0.25 * max(1, len(result.missing_cases)))
```

`max(1, ...)` tồn tại vì một phiên bản trước của công thức này tính `1.0 - 0.25 * 0 == 1.0` bất cứ khi nào mô hình trả về `handled=False` nhưng danh sách `missing_cases` rỗng — âm thầm ghi đè phán quyết “không xử lý được” của chính mô hình bằng một điểm tuyệt đối. Danh sách rỗng ở đây có nghĩa “chưa xác định”, không phải “không bị trừ điểm”. Đây là một lỗi nhỏ, nhưng đúng kiểu lỗi mà một lượt kiểm tra nhanh trên happy-path sẽ bỏ sót, và nó trực tiếp làm suy yếu tuyên bố correctness trung tâm của dự án nếu để sót lại.

4.4 Tính xác định được đo, không giả định

`temperature=0` + seed cố định + giải mã ràng buộc **không** đảm bảo lặp lại chính xác từng bit dưới một serving engine có batching — tính không kết hợp của số học dấu phẩy động qua các tổ hợp batch khác nhau là một hiện tượng có thật, đã được ghi nhận trong serving LLM, không phải mối lo lý thuyết.

`experiments/consistency_test.py` đo trực tiếp điều này thay vì khẳng định cấu hình làm cho hệ thống có tính xác định.



Hình 4: Phương pháp đo consistency: cùng một mẫu cố định được chấm 100 lần dưới cả serving không batching và có batching. Kết quả trên 4 mẫu dễ và 4 mẫu cố tình khó (có lỗi / trường hợp biên): tỷ lệ exact-match = 1.0 ở mọi điều kiện, phương sai bằng 0 trên mọi breakdown theo từng check. Xem `docs/thesis/results.md` để biết các lưu ý về phạm vi (cơ chế batching cụ thể của llama.cpp, một mức concurrency).

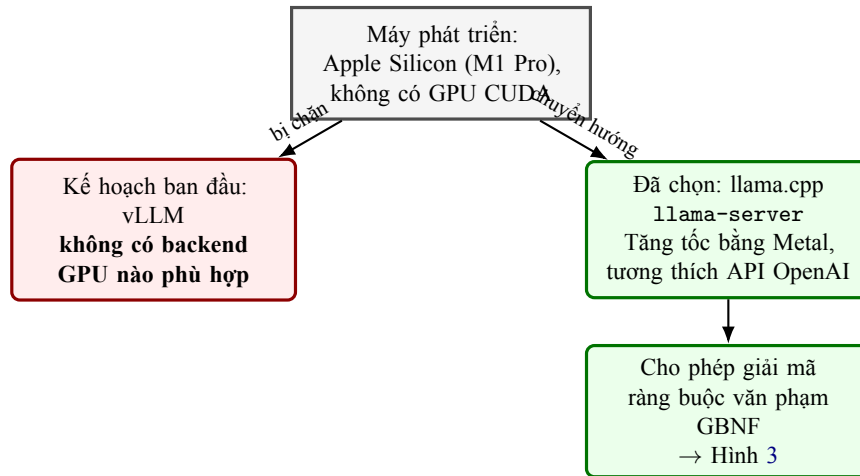
4.5 Triển khai xoay quanh giới hạn phần cứng, không bắt chấp nó

Kế hoạch ban đầu giả định dùng vLLM, serving engine phổ biến nhất trong tài liệu LLM-as-judge mà dự án này dựa vào. Máy phát triển là Apple Silicon (M1 Pro) không có GPU hỗ trợ CUDA — vLLM không có backend GPU nào được hỗ trợ trên nền tảng này.

5 Kết quả định lượng

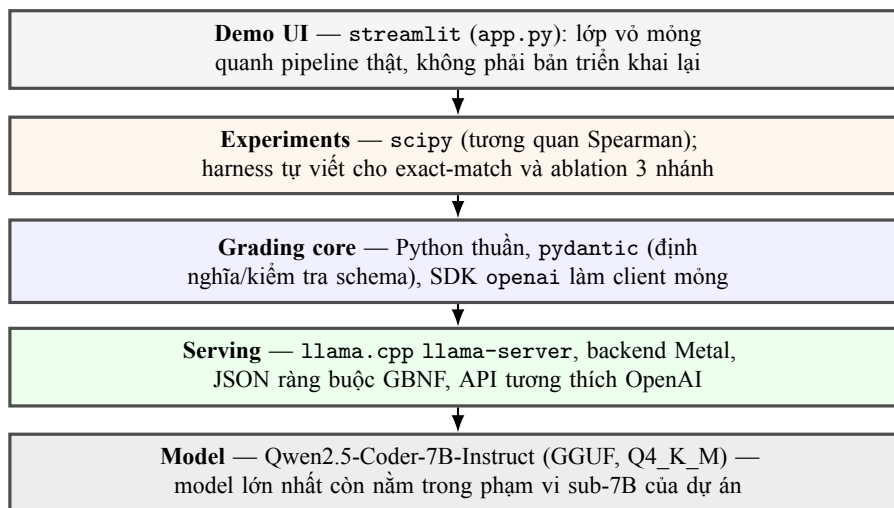
Thực nghiệm	Thiết lập	Kết quả
Consistency, không batching	4 mẫu × 100 lần chạy, <code>--parallel 1</code>	exact-match = 1.0, cả 4 mẫu
Consistency, có batching	Cùng 4 mẫu × 100 lần chạy, <code>--parallel 4</code> , 8 đồng thời	exact-match = 1.0, cả 4 mẫu
Consistency, mẫu khó	4 bài nộp có lỗi/trường hợp biên × 100 lần, cả hai điều kiện	exact-match = 1.0, cả 4 mẫu
Ablation (căn cứ kiến trúc)	65 mẫu, 3 nhánh, Spearman ρ so với nhãn tham chiếu	0.60 → 0.85 → 0.88
Parse fail, các nhánh free-form	65 mẫu, cả hai điều kiện free-form	0/65

Bảng 1: Các con số chính, đều lấy từ `docs/thesis/results.md` (không tính lại ở đây).



Hình 5: Việc chuyển serving engine bị ép buộc bởi phần cứng, nhưng đó cũng chính là điều làm cho giải mã ràng buộc theo schema ở Hình 3 khả thi trên máy này — `src/grader/llm_client.py` không cần thay đổi gì ngoài `base_url` vì cả hai engine đều phơi bày API tương thích OpenAI.

6 Stack công nghệ



Hình 6: Stack phân lớp, lớp trên sử dụng lớp dưới.

7 Thực hành kỹ thuật đáng chú ý

- **Chung một đường code giữa bộ sinh dữ liệu và pipeline thật:** `sandbox.run_submission()` là hàm giống hệt mà `data/build_dataset.py` dùng để sinh nhãn tham chiếu, nên pipeline chấm điểm và bộ dữ liệu dùng để kiểm chứng nó không thể âm thầm lệch nhau theo thời gian.
- **Dữ liệu sinh ra, không commit:** `data/dataset.json` được sinh từ `data/problems.py` qua `data/build_dataset.py` và bị gitignore — nguồn sự thật là bộ sinh, không phải một file JSON cũ có thể lỗi thời.
- **Lưu lại các lần so sánh, không ghi đè kết quả:** mỗi lần chạy lại ở một kích thước model hoặc dataset mới đều giữ lại kết quả trước đó (`results/archive_1.5b/`, `results/ablation_33samples_7b.json`) để các tuyên bố trước/sau có thể kiểm chứng được, không chỉ là khẳng định suông.

- **Hạn chế đã biết được nêu ngay tại nơi dữ liệu sống, không giấu đi:** `data/script.md` ghi rõ, ngay trong thư mục của bộ dữ liệu, rằng nhãn `efficiency/code_style` là tự gán nhãn một người, không phải ground truth độc lập — mọi bảng kết quả dùng các nhãn này đều nhắc lại lưu ý đó.

8 Những gì cố tình nằm ngoài phạm vi (và vì sao đó là một quyết định)

- **Fine-tuning (LoRA):** được xây như một phương án dự phòng, có cổng go/no-go. Vì prompting + giải mã có cấu trúc đã vượt ngưỡng consistency và agreement của dự án (Bảng 1), giai đoạn fine-tuning bị bỏ qua — ghi nhận như một phát hiện (`docs/roadmap.md`, Phase 4), không phải một tính năng đang dở.
- **Kiểm chứng bằng giám khảo con người thật:** các con số agreement hiện tại là so với nhãn tham chiếu dựa trên luật, không bao giờ được gọi là “đồng thuận với giám khảo con người” ở bất kỳ đâu trong tài liệu repo (xem lưu ý trong `data/script.md`). Việc tuyển một giám khảo độc lập là một bước tiếp theo đã được lên kế hoạch nhưng chưa bắt đầu (`docs/roadmap.md` T3.2), tách bạch rõ ràng khỏi các con số không phụ thuộc vào nó.